CrossTrust: A Blockchain-Based Trust Architecture for Cross-Organization AI Agents over MCP and A2A

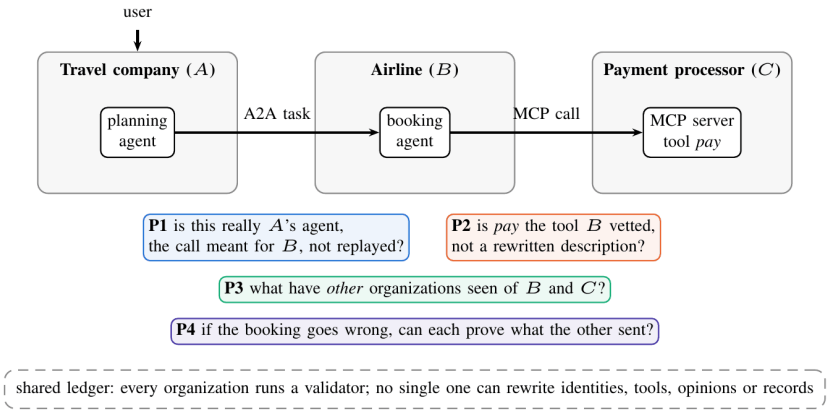
Hazael S. A. Jumonji hazael.soledade@sjtu.edu.cn · hsoledade.github.io

M.Sc. student, Shanghai Jiao Tong University

Joint work with my advisor, Prof. Yue Wu, and Dr. Jianan Hong (SJTU). Manuscript in preparation, available on request.

The problem

AI agents run by different organizations increasingly call each other through the Agent2Agent protocol (A2A) and each other's tools through the Model Context Protocol (MCP). Neither protocol lets a receiver verify **who** is calling, whether the agent or tool is **the one it vetted**, what **other organizations** have observed of it, or **what happened** in an interaction. Existing defenses answer these questions through one party that every organization must trust, or through a ledger that anyone can write to.



Key idea

What must be decentralized is not storage but **the authority to write trust state**. In CrossTrust, every call carries one backward-compatible signed field, checked against identities, tool hashes, reputation and evidence kept on a permissioned Byzantine-fault-tolerant ledger (QBFT, one validator per organization). Each write belongs to one role, and only a quorum of organizations can override it. No single organization, including the one that deployed the system, can rebind an identity, swap a tool without notice, move another agent's reputation with identities it creates, or rewrite the record of an interaction.

My role

Under the supervision of Prof. Yue Wu and with Dr. Jianan Hong's guidance, I designed, implemented and evaluated the system: the smart contracts and their quorum governance (Hyperledger Besu, Solidity), a Python trust module that plugs into existing MCP and A2A software, multi-agent testbeds, a benchmark of 18 systems run from their public code, and machine-checked proofs.

Results

- **Identity.** It stops all seven identity attacks on both protocols; none of the 18 other systems stops more than five.
- **Tool integrity.** It stops every tool swap by another party, even on a client's first contact. With a real LLM, a swapped poisoned tool is obeyed in 12% of the decided cases (90 of 100) without a defense, and in none with CrossTrust.
- **Reputation.** Fake identities do not help an attacker: faking a good reputation takes four independent operators, while in a public on-chain registry one operator's own identities suffice.
- **Evidence.** It catches every record that one party drops or falsifies, because the counterpart anchors the same record.
- **Proofs.** Reputation results machine-checked in **Lean 4**; the signed field each call carries verified in **Tamarin**.
- **Cost.** Under 3 ms of checks per call, 0.2% of an LLM call. A central server with the same cache is about 0.5 ms faster and serves twice the load: the price of having no single trusted party.

What I want to do next

- **Accountability for agent ecosystems:** evidence of what an agent did that still holds when the operator who ran it is dishonest, auditable by a third party without trusting either side.
- **Reputation that resists collusion:** aggregation rules whose robustness to Sybils and colluding operators is backed by a proof, and cheap enough to check on every call.
- **The tool supply chain of LLM agents:** defenses against poisoned or silently changed tools that work across vendors and do not rely on a single registry.